Escuela Superior Politécnica del Litoral

Facultad de Ingeniería Ciencias Naturales Y Matemáticas

Diseño de Software

Taller05-Patrones-Creacionales

Grupo #6

Integrantes:

Zuñiga Murillo Ezequiel Alfredo
Recalde Bravo Ashley Dayanna
Arboleda Torres Camilo Vicente
Jean Marcos Benites Segura

Paralelo:

107

Tabla de contenido

Sección A.-....3
Sección B:....7
Diagrama de clases:....7
Diagrama de Secuencias....8
Singleton....8
Factory Method....9
Sección C:....9

Sección A.-

1. **Patrones Identificados:**
   De acuerdo al enunciado del problema se debe mantener el foco en dos piezas: Acceso a datos y Creación de productos; por ello a continuación se detalla el patrón que cumple de acuerdo al sistema del problema.

Acceso a datos	Cumple el patrón de Singleton	De acuerdo al sistema que menciona que “solo exista una instancia activa en todo el sistema, evitando duplicación de recursos y posibles conflictos de acceso a los datos ”
Creación de productos	Cumple el patrón de Factory Method	De acuerdo al sistema que menciona que “decide dinámicamente qué tipo de objeto construir según el tipo solicitado ”, además “añadir nuevos tipos de productos sin modificar el resto del código ”

2. **SUPUESTOS:**

- Exista una unica base datos central y general
- El sistema puede recibir solicitudes concurrentes, por lo que el patron Singleton debe ser seguro ante hilos

- Cada tipo de producto tiene su propia clase creadora; por lo que si un tipo nueva se agrega creando una subclase de producto y una de creador, sin que se modifique lo existente.
- Un producto digital no maneja inventario fisico: su stock es ilimitado o no aplica.

3. PATRÓN 1: SINGLETON, que pertenece a Acceso a datos.

Objetivo: Asegura que una clase tenga una sola instancia en toda la aplicación y puede proporcionar un punto de acceso global.
Motivación: Todas las funciones del sistema deben trabajar sobre la misma fuente de datos (registrar categorías, crear productos, consultar inventario y precios). Si se crea una propia conexión, esto causará duplicación de recursos y podrían generarse conflictos de acceso e intermitencias; por eso solo debe haber un único gestor, esto permitirá establecer conexión y atender todas las consultas y actualizaciones.
Clase del sistema: DatabaseConnection que implementa IAccesoDatos
Consecuencias:

Ventajas	Desventajas
- Controla a la única instancia - Ahorra recursos porque solo se hace una sola conexión - Se configura desde un solo punto	- Dificulta las pruebas individuales ya que se implementa un estado global - Debe sincronizarse en un entorno multihilo - Tiende a ocultar las dependencias y generar un acoplamiento.

Relación con SOLID:

PRINCIPIO SOLID	CUMPLE	JUSTIFICACIÓN
SRP	Si cumple	La clase solo tiene una única responsabilidad:

		gestionar la conexión y el acceso a la base de datos.
OCP	Cumple de manera parcial	Si la clase implementa una interfaz de acceso a datos, entonces puede extenderse; pero la clase no es extensible por herencia.
DIP	Cumple de manera parcial	Si los clientes llaman a getIstance(), deben depender de una clase concreta.
LSP	No cumple	No se sustituye ninguna jerarquía.
ISP	No cumple	La interfaz IAccesoDatos se mantiene pequeña.

4. *PATRÓN 2: FACTORY METHOD que pertenece a la creación de productos.*

Objetivo: Implementar una interfaz para crear un objeto, esto hará que las subclases decidan qué clase instanciar.
Motivación: El sistema indica que se comercializa productos físicos como peso y dimensiones y digitales como tamaño y formato; además debe agregarse nuevos tipos sin que se modifique el código que ya existe. Este patrón se basa en la decisión de crear un punto que se pueda extender.
Clases Producto, ProductoFisico, ProductoDigital, Creator, ConcreteCreatorFisico y ConcreteCreatorDigital.

Consecuencias:

Ventajas	Desventajas
- Dependen del acoplamiento del cliente y los productos. - Un tipo nuevo requiere una subclase	- El número de clases aumenta - Tiene una mayor complejidad - El cliente debe elegir el creador

de producto y su creador.	correcto.

Relación con SOLID

PRINCIPIO SOLID	CUMPLE	JUSTIFICACIÓN
SRP	Si cumple	Cada creador tiene la única responsabilidad de construir un tipo de productos
OCP	Si cumple	Se extiende agregando nuevas clases sin que se modifique el código existente.
DIP	Si cumple	El cliente depende de las abstracciones de Producto y ProductoCreator.
LSP	Si cumple	Cualquier subtipo de Producto puede usarse donde se espera un Producto.
ISP	Si cumple	El creador expone solo lo necesario para fabricar.

Sección B:

Diagrama de clases:

Hecho en VisualParadigm

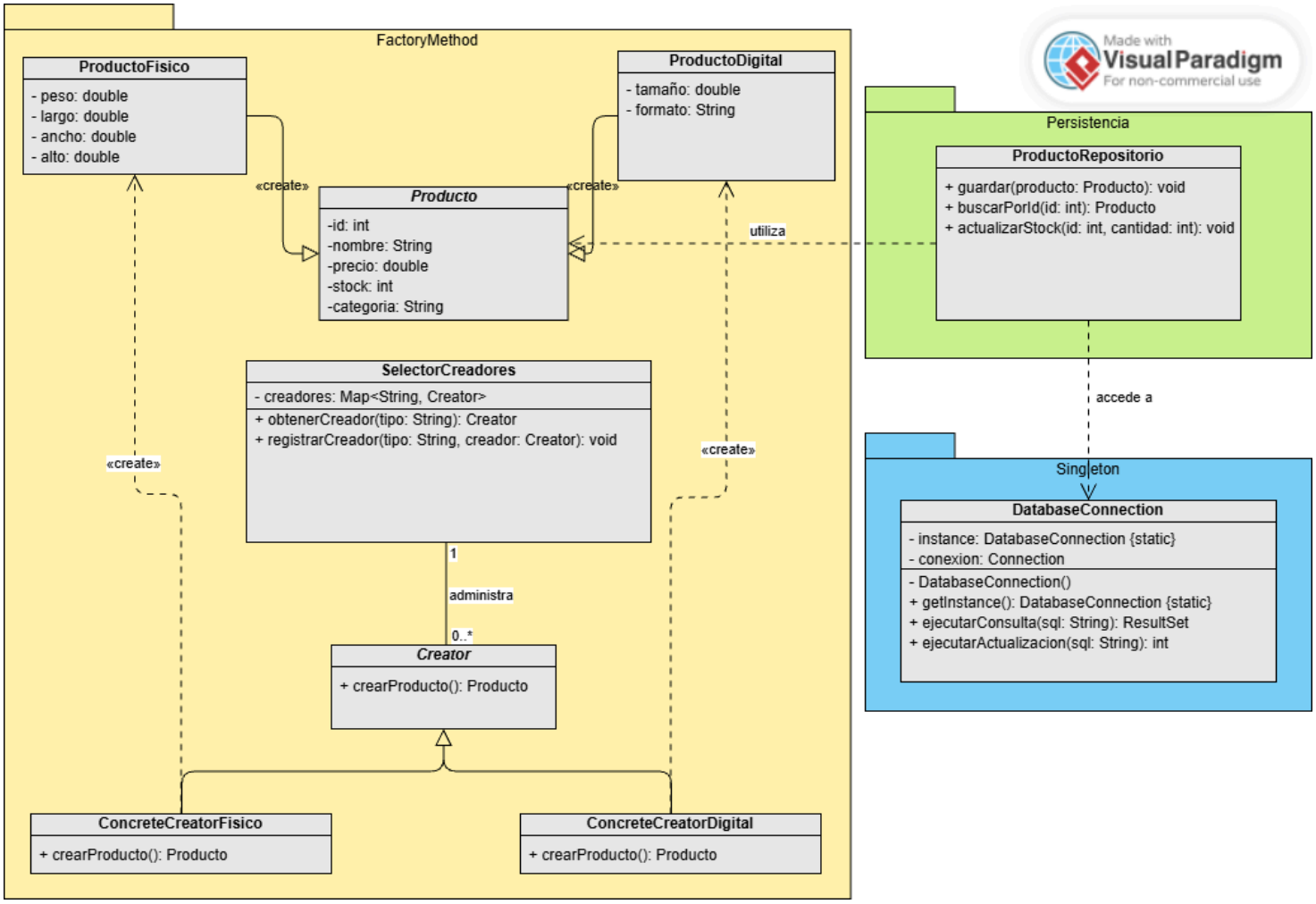
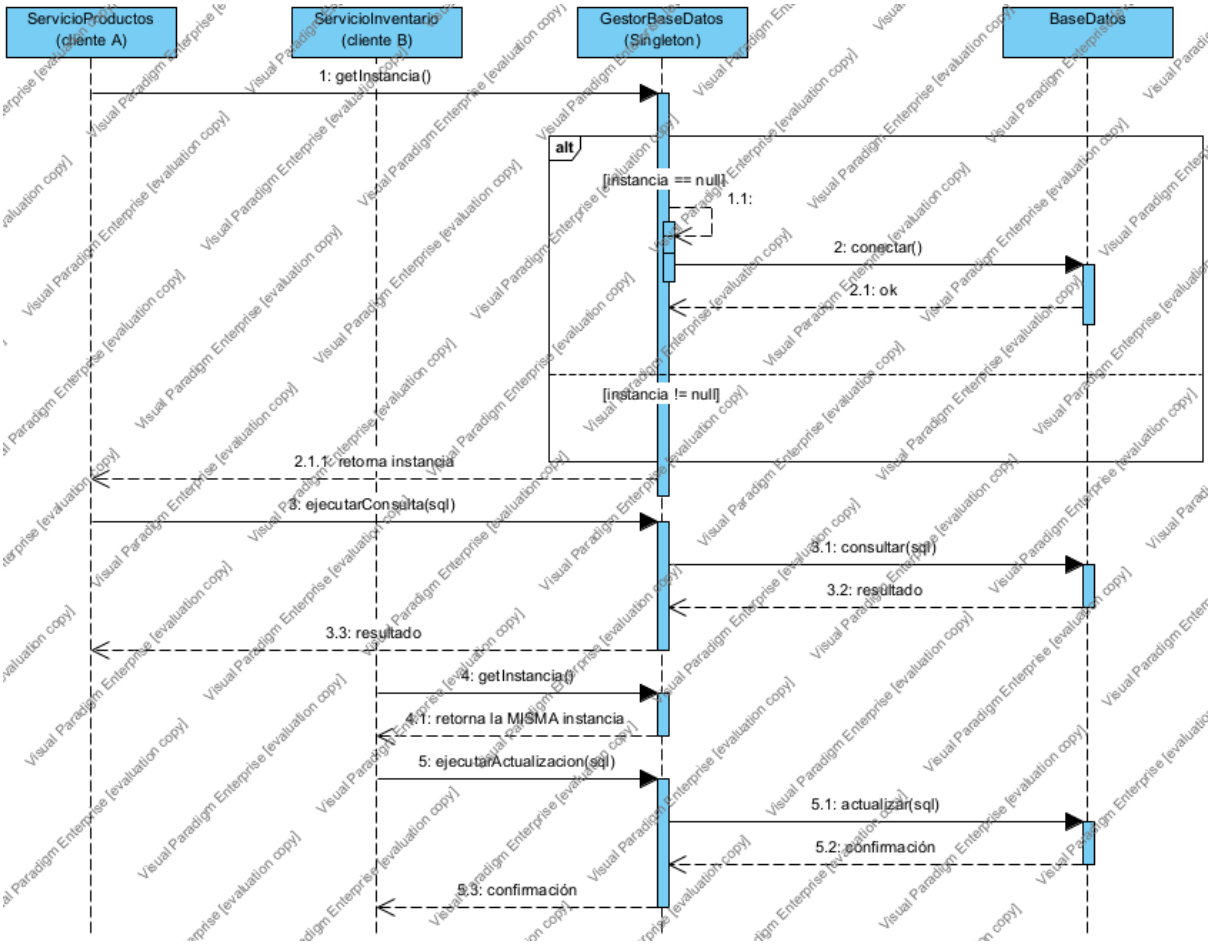


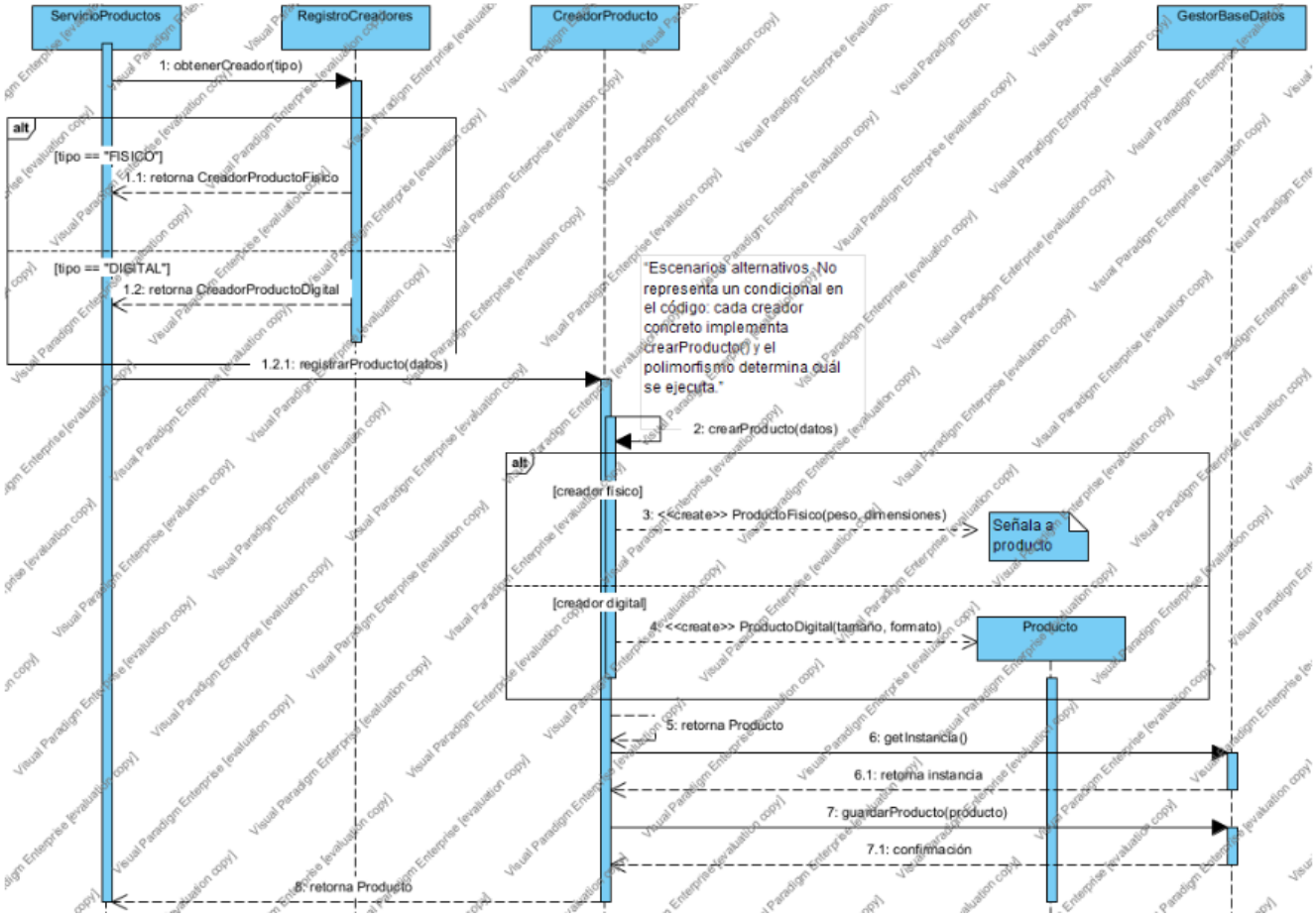
Diagrama de Secuencias

Hecho en VisualParadigm

Singleton



Factory Method



Sección C: Código en JAVA

REGISTRO DINAMICO

```java
import factorymethod.*;
import java.util.HashMap;
import java.util.Map;
import singleton.dbcentral;

public class Main {

    private static final Map<String, ProductoCreator> creadores = new HashMap<>();

    static {
        creadores.put("FISICO", new FisicoCreator());
        creadores.put("DIGITAL", new DigitalCreator());
    }

    public static Producto crearYRegistrarProducto(String tipo, String id, String nombre,
                                                   double precio, Map<String, Object> atributos) {

        ProductoCreator creador = creadores.get(tipo.toUpperCase());
        if (creador == null) {
            throw new IllegalArgumentException("Tipo de producto no soportado: " + tipo);
        }
        return creador.registrarProducto(id, nombre, precio, atributos);
    }

    Run main | Debug main | Run | Debug
```

EJECUCION POR CONSOLA

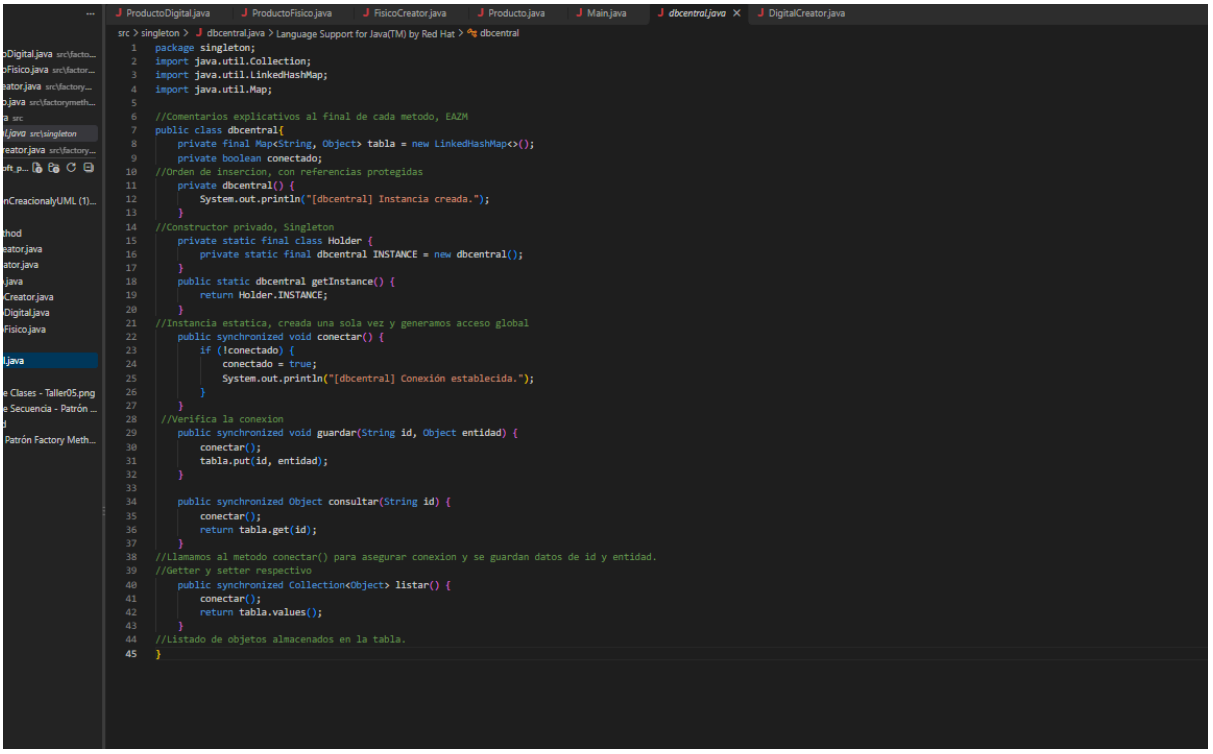
```
return creador.registrarProducto(id, nombre, precio, atributos);

Problems  Output  Debug Console  Terminal  Ports

Listening on 50336
User program running
[dbcentral] Instancia creada.
--- Creaci□n Din□mica de Productos ---
[dbcentral] Conexi□n establecida.
[FISICO] Monitor 24 pulgadas (id=P001, $150,00) peso=3.5kg, dimensiones=54x32x5 cm
[DIGITAL] Licencia de Antivirus (id=P002, $45,50) tama□o=15.0MB, formato=EXE

--- Verificaci□n en la Base de Datos Central ---
Total de productos guardados: 2
Consultando P002: [DIGITAL] Licencia de Antivirus (id=P002, $45,50) tama□o=15.0MB, formato=EXE
User program finished
```

APLICACION DE PATRON CREACIONAL SINGLETON EN CLASE DBCENTAL.JAVA

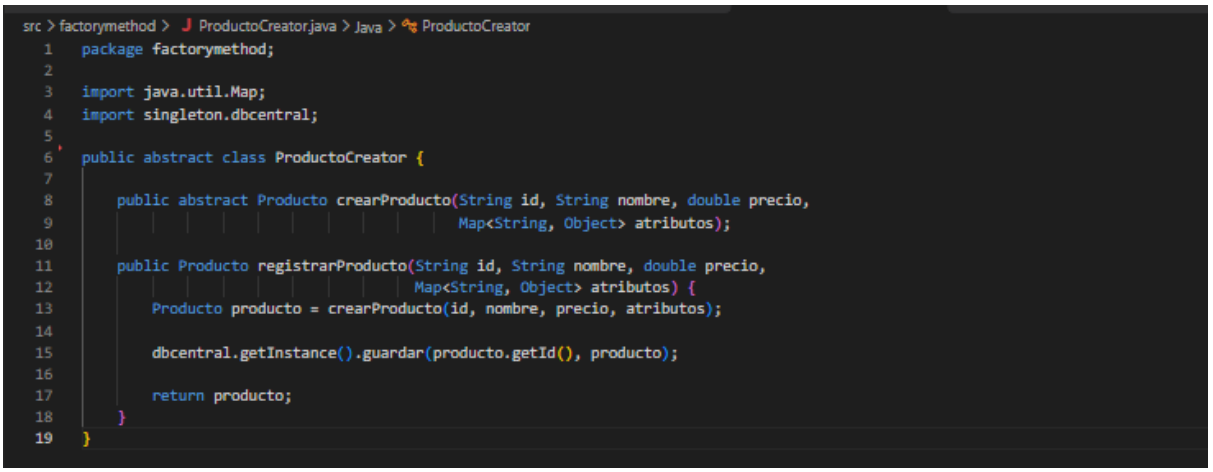


```java
package singleton;
import java.util.Collection;
import java.util.LinkedHashMap;
import java.util.Map;

//Comentarios explicativos al final de cada metodo, EAZM
public class dbcentral{
    private final Map<String, Object> tabla = new LinkedHashMap<>();
    private boolean conectado;
//Orden de insercion, con referencias protegidas
    private dbcentral() {
        System.out.println("[dbcentral] Instancia creada.");
    }
//Constructor privado, Singleton
    private static final class Holder {
        private static final dbcentral INSTANCE = new dbcentral();
    }
    public static dbcentral getInstance() {
        return Holder.INSTANCE;
    }
//Instancia estatica, creada una sola vez y generamos acceso global
    public synchronized void conectar() {
        if (!conectado) {
            conectado = true;
            System.out.println("[dbcentral] Conexión establecida.");
        }
    }
 //Verifica la conexion
    public synchronized void guardar(String id, Object entidad) {
        conectar();
        tabla.put(id, entidad);
    }

    public synchronized Object consultar(String id) {
        conectar();
        return tabla.get(id);
    }
//Llamamos al metodo conectar() para asegurar conexion y se guardan datos de id y entidad.
//Getter y setter respectivo
    public synchronized Collection<Object> listar() {
        conectar();
        return tabla.values();
    }
//Listado de objetos almacenados en la tabla.
}
```

APLICACION DE PATRON CREACIONAL FACTORY METHOD EN CLASE PRODUCTOCREATOR.JAVA



```java
package factorymethod;

import java.util.Map;
import singleton.dbcentral;

public abstract class ProductoCreator {

    public abstract Producto crearProducto(String id, String nombre, double precio,
                                           Map<String, Object> atributos);

    public Producto registrarProducto(String id, String nombre, double precio,
                                      Map<String, Object> atributos) {
        Producto producto = crearProducto(id, nombre, precio, atributos);

        dbcentral.getInstance().guardar(producto.getId(), producto);

        return producto;
    }
}
```